

Resumen Técnico - EMA200 App

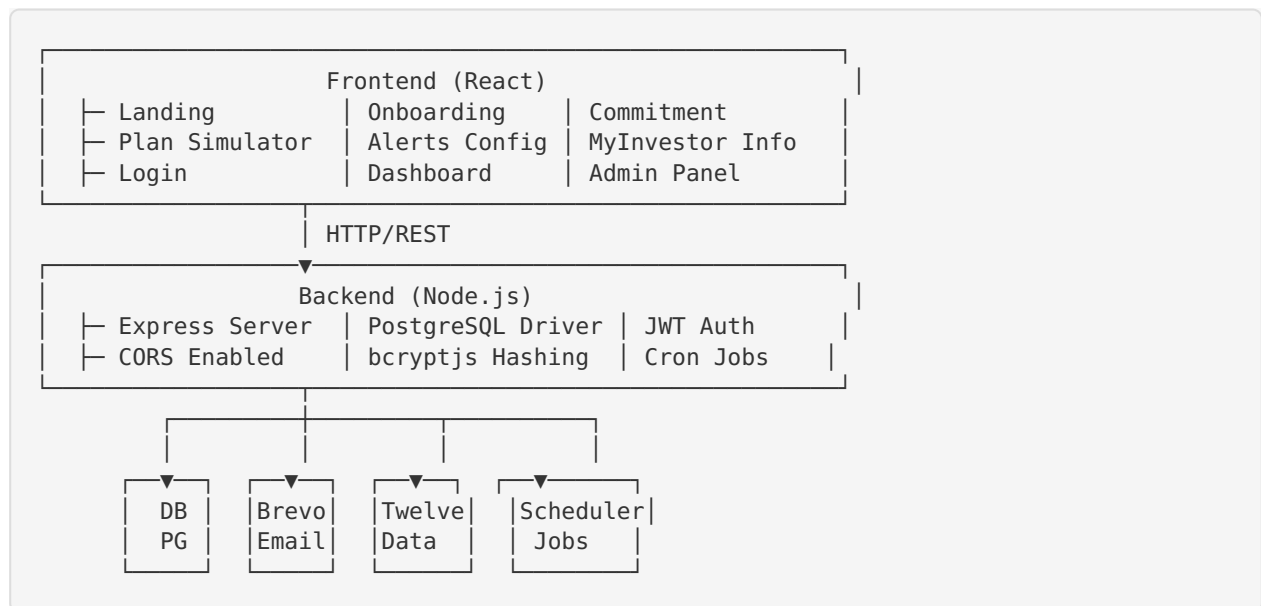
Descripción General

Aplicación web full-stack para inversores españoles que automatiza una estrategia de jubilación basada en el EMA200 del S&P 500 y tres fondos de MyInvestor.

Características principales:

- Registro y autenticación con PIN de 4 dígitos
- Panel administrativo para aprobar usuarios
- Simulador de jubilación con 3 escenarios
- Alertas automáticas por email (Brevo)
- Datos de mercado en tiempo real (Twelve Data)
- Rotación automática a dividendos
- Historial y estadísticas

Arquitectura



Stack Tecnológico Detallado

Frontend

- **React 19.2.7**: UI library
- **Vite 8.1.1**: Build tool (ultrafast)
- **Tailwind CSS 4.3**: Utility-first CSS
- **React Router DOM 7.18**: Routing
- **Zustand 5.0**: State management
- **Axios 1.18**: HTTP client
- **date-fns 4.4**: Date utilities

Características:

- Mobile-first responsive design
- Client-side routing sin refresh
- State management minimalista
- Formularios con validación
- Copy-to-clipboard para ISINs

Backend

- **Node.js 18+:** Runtime
- **Express 4.18:** Web framework
- **PostgreSQL:** Base de datos relacional
- **pg 8.9:** PostgreSQL driver
- **jsonwebtoken 9.0:** JWT auth
- **bcryptjs 2.4:** Password hashing
- **axios 1.3:** HTTP client (Twelve Data)
- **node-cron 3.0:** Scheduled tasks
- **cors 2.8:** CORS middleware

Características:

- RESTful API design
- JWT tokens (7 días expiración)
- PIN hashing con bcryptjs
- Cron jobs automáticos
- Email transaccional con Brevo

Base de Datos (PostgreSQL)**Tablas:**

```

users
❑ id (PK)
❑ email (UNIQUE)
❑ name
❑ birth_date
❑ monthly_contribution
❑ pin_hash
❑ status (pending, approved, rejected)
❑ approved_at
❑ created_at
❑ updated_at

ema200_history
❑ id (PK)
❑ date
❑ price
❑ ema200
❑ signal (buy/null)
❑ created_at

email_log
❑ id (PK)
❑ user_id (FK)
❑ email_type
❑ subject
❑ sent_at
❑ status

rotation_history
❑ id (PK)
❑ user_id (FK)
❑ rotation_year (1-5)
❑ status (pending, completed, paused)
❑ percentage
❑ completed_at
❑ created_at

```

Seguridad

Autenticación

- Email + PIN de 4 dígitos
- JWT tokens con expiración 7 días
- PIN hasheado con bcryptjs (salt rounds: 10)
- Recuperación de PIN por email

Autorización

- Token JWT en header `Authorization: Bearer <token>`
- Admin email verificado en header para panel
- Usuarios solo ven sus propios datos
- Admin solo accede con email correcto

CORS

- Permitido desde FRONTEND_URL
- Métodos: GET, POST, PUT, DELETE
- Headers: Content-Type, Authorization

Sistema de Emails

Proveedor: Brevo (ex-Sendinblue)

Emails enviados automáticamente:

1. Registro Pendiente

- Trigger: Después del registro
- Receptor: Usuario
- Contenido: Confirmación pendiente de aprobación

2. Aprobación de Cuenta

- Trigger: Admin aprueba usuario
- Receptor: Usuario
- Contenido: Bienvenida y próximos pasos

3. Motivación Mensual

- Trigger: Día 1 de cada mes a las 08:00 AM
- Receptor: Todos los usuarios aprobados
- Contenido: Recordatorio de aportación mensual

4. Señal de Compra (EMA200)

- Trigger: S&P 500 toca EMA200 ($\text{precio} \leq \text{ema200}$)
- Receptor: Todos los usuarios aprobados
- Contenido: Instrucciones paso a paso para comprar

5. Rotación a Dividendos

- Trigger: 5-1 años antes de jubilación (anual)
- Receptor: Usuarios elegibles
- Contenido: Porcentaje a traspasar + instrucciones
- Variante: Pausa si hay crash ($\text{precio} \leq \text{ema200}$)

6. Recuperación de PIN

- Trigger: Usuario olvida PIN
- Receptor: Usuario
- Contenido: Nuevo PIN de 4 dígitos

Cron Jobs (Tareas Automáticas)

Ejecutados por `node-cron` en el servidor:

08:00 AM (Diario)

```
Función: dailyEMA200Check()
├─ Obtiene precio actual del S&P 500 (Twelve Data)
├─ Calcula EMA200 de 52 semanas
├─ Si  $\text{precio} \leq \text{ema200}$ :
│   └─ Envía "Señal de Compra" a todos los usuarios aprobados
└─ Registra datos en ema200_history
```

01:00 AM del día 1 (Mensual)

```

Función: monthlyMotivation()
├─ Obtiene lista de usuarios aprobados
├─ Envía email "Motivación Mensual" a cada uno
└─ Registra en email_log

```

09:00 AM (Diario)

```

Función: checkRotationAnniversary()
├─ Para cada usuario aprobado:
│   ├── Calcula años hasta jubilación
│   ├── Si yearsUntilRetirement ≤ 5:
│   │   ├── Verifica estado de mercado
│   │   ├── Si normal (precio > ema200):
│   │   │   └─ Envía instrucción de rotación
│   │   ├── Si crash (precio ≤ ema200):
│   │   │   └─ Envía aviso de pausa
│   └─ Registra en rotation_history
└─ Continúa después de jubilación si rotación incompleta

```



Simulador de Jubilación

Cálculos:

```

// Input
monthlyAmount = €500
annualReturn = 10% (escenario histórico)
yearsUntilRetirement = 30

// Proceso
monthlyReturn = (1 + 0.10)^(1/12) - 1 = 0.00797...
patrimonio = 0
for (i = 0 to months) {
    patrimonio = patrimonio * (1 + monthlyReturn) + monthlyAmount
}

// Resultado
patrimonioEstimado = €1,145,666

// Dividendos (3.5% anual)
dividendosAnuales = patrimonio * 0.035 = €40,098
dividendosMensual = €3,341

// Retenciones fiscales (españolas)
if (dividendosAnuales <= 6000) retención = 19%
else if (dividendosAnuales <= 50000) retención = 21%
else retención = 23%

dividendosNetos = dividendosAnuales * (1 - retención)

```

Flujo de Pantallas

```

Landing (sin scroll, minimalista)
  ↓ "Quiero empezar"
Onboarding (formulario)
  ↓ Siguiente
Commitment (aceptar compromiso)
  ↓ Aceptar
Plan (simulador + 3 escenarios)
  ↓ Continuar
Alerts (confirmación de alertas)
  ↓ Continuar
Invest (MyInvestor + ISINs)
  ↓ Ir al Dashboard
Dashboard (principal, tabs: overview, fondos, historial, settings)

Flujo alternativo:
Login (email + PIN)
  ↓
Dashboard (si aprobado)
  ├── Resumen
  ├── Mis Fondos (ISINs copiables)
  ├── Historial EMA
  └── Configuración

Admin Panel (/admin)
  ├── Pendientes (aprobar/rechazar)
  ├── Aprobados (tabla de usuarios)
  └── Estadísticas

```

Flujo de Datos

Registro

```

Frontend (Onboarding)
  ↓ POST /api/auth/register
Backend (authAPI.register)
  ├── Valida email, PIN, fechas
  ├── Hashea PIN con bcryptjs
  ├── Inserta en BD con status='pending'
  ├── Envía email "Registro Pendiente" (Brevo)
  └── Retorna usuario (sin PIN)
Frontend (Commitment)

```

Aprobación de Usuario

```

Admin Panel
  ↓ Click "Aprobar"
Backend (adminAPI.approveUser)
  ├── Verifica email de admin
  ├── Actualiza status='approved'
  ├── Set approved_at=now
  ├── Envía email "Bienvenida" (Brevo)
  └── Retorna usuario
BD → email_log

```

Login

```
Frontend (Login)
  ↓ POST /api/auth/login (email, PIN)
Backend (authAPI.login)
  ├── Busca usuario por email
  ├── Verifica status='approved'
  ├── Compara PIN con hash
  ├── Si todo OK: genera JWT
  ├── Calcula yearsUntilRetirement
  └── Retorna token + userData
Frontend
  ├── Guarda token en localStorage
  ├── Set isLoggedIn=true
  └── Redirige a /dashboard
```

Obtener Fondos

```
Frontend (Invest)
  ↓ GET /api/funds/list (requiere token)
Backend
  ├── Verifica JWT
  └── Retorna array de 3 fondos con ISIN
Frontend
  ├── Muestra tarjetas
  └── Click "Copiar" → navigator.clipboard.writeText(ISIN)
```



Datos de Mercado

Proveedor: Twelve Data API

Endpoints:

```
GET https://api.twelvedata.com/time_series
  ├── symbol: SPY (S&P 500)
  ├── interval: week (semanal)
  ├── outputsize: 52 (últimas 52 semanas)
  └── apikey: 37df2fa94b664d9b969beb200222fcac
```

Datos obtenidos:

- Precio actual
- OHLCV (Open, High, Low, Close, Volume)
- Historial de 52 semanas

Cálculo EMA:

```
// EMA = Exponential Moving Average
k = 2 / (period + 1) = 2/201 para 200 períodos
ema = price[period-1]
for (i = period-2; i >= 0; i--) {
  ema = price[i] * k + ema * (1 - k)
}
```

Índices de Base de Datos

Optimización de queries:

```
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_status ON users(status);
CREATE INDEX idx_ema200_date ON ema200_history(date);
CREATE INDEX idx_email_log_user ON email_log(user_id);
```

Performance

Frontend

- **Lazy loading:** Componentes cargan bajo demanda
- **Code splitting:** React Router automático
- **Minification:** Vite build automático
- **Tailwind JIT:** CSS generado dinámicamente
- **Local storage:** Cache de token

Backend

- **Connection pooling:** PG pool
- **Índices BD:** Búsquedas rápidas
- **JWT stateless:** Sin sesiones
- **Async/await:** No bloquea evento loop

Convenciones de Código

Backend

- Arrow functions
- Async/await (no callbacks)
- try/catch para errores
- Separación de concerns (routes, services, jobs)
- Nombres descriptivos en español y inglés

Frontend

- Componentes funcionales (no clase)
- Hooks (useState, useEffect)
- Custom hooks si es reutilizable
- Zustand para estado global
- Tailwind para styles
- Nombres camelCase

Testing

Ejemplos de pruebas manuales:


```
# Register usuario
curl -X POST http://localhost:5000/api/auth/register \
-H "Content-Type: application/json" \
-d '{
  "email": "test@example.com",
  "name": "Test",
  "birthDate": "1980-05-15",
  "monthlyContribution": 500,
  "pin": "1234"
}'

# Login
curl -X POST http://localhost:5000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email": "test@example.com", "pin": "1234"}'

# Obtener perfil (requiere token)
curl -H "Authorization: Bearer [token]" \
http://localhost:5000/api/users/profile
```

Archivos Clave

```
backend/
├─ config.js           # Credenciales y configuración
├─ index.js            # Servidor Express
├─ routes/auth.js      # Autenticación
├─ routes/users.js     # Perfil y simulaciones
├─ routes/funds.js     # ISINs y historial
├─ routes/admin.js     # Panel administrativo
├─ services/emailService.js # Envío de emails
├─ jobs/cronJobs.js    # Tareas automáticas
└─ scripts/initDb.js   # Inicialización BD

frontend/src/
├─ App.jsx             # Router principal
├─ main.jsx            # Entry point
├─ index.css           # Tailwind CSS
├─ stores/authStore.js # Estado global
├─ services/api.js     # Cliente HTTP
└─ pages/
  ├─ Landing.jsx
  ├─ Onboarding.jsx
  ├─ Commitment.jsx
  ├─ Plan.jsx
  ├─ Alerts.jsx
  ├─ Invest.jsx
  ├─ Login.jsx
  ├─ Dashboard.jsx
  └─ AdminPanel.jsx
```

Última actualización: Julio 2026

Versión: 1.0.0

Estado: Production Ready